QUALITY ASSURANCE TESTING DOCUMENTATION

(QATD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Content

Document Control 3

PRELIMINARY ROUND (Test Strategy & Planning) **4**

1. Scope & Requirements Traceability 4
2. Risk Assessment & Mitigation Strategy 5
3. Test Environment & Execution Strategy 6
4. CI/CD Release Thresholds & Automation Gates 8
5. Test Case Specifications (Drafts) 9
6. Test Strategy & Plan Sign-Off 10

© UMHackathon 2026

Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	UMHackathon 2026 - Team ShakeHand
Team Repo URL	*https://github.com/zyl-1106/UMHackathon2026-TeamShakeHand.git*
Project Board URL	*https://docs.google.com/spreadsheets/d/1Tsfuj0bZqG78feAPWngLBDI1IRDVehi1_7gvD98iNE8/edit?usp=sharing*
Live Deployment URL	**Victim Portal:** *https://zai-flood-app-yzvkbp5wys8ol3fhtgau4m.streamlit.app/* **Rescue Dashboard:** *https://zai-rescue-app-cgihct8c7q7fwijby2ysvr.streamlit.app/*

Objective

The primary objective is to ensure that the **Z.AI Emergency Portal** reliably handles victim SOS submissions, accurately parses data for priority assignment, and ensures seamless synchronization with the Rescue Command Center via simulated mesh protocols with CI/CD checkpoints.

© UMHackathon 2026

Email: umhackathon@um.edu.my

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

1.1 In-Scope Core Features

- **Victim SOS Submission & AI Parsing:** User-friendly emergency form with automated priority categorization.
- **Agentic Priority Engine:** Automated logic to escalate "Chest Level" or clustered signals.
- **Command Center Sync & Dispatch:** Real-time dashboard for rescuers to update mission status (Awaiting, Pending, Rescued).

1.2 Out-of-Scope

- **Physical Mesh Network Transmission:** This project focuses on the software logic and simulated protocol for data relay. Physical deployment of radio nodes or hardware-based signal routing is out of scope.
- **Hardware Sensor Integration:** Real-time hardware triggers, such as physical water depth sensors or automated IoT beacons, will not be integrated or tested. Input is limited to user-reported data through the web portal,
- **External API Integrations:** Direct integration with external third-party services, such as live Google Maps API (outside of basic display), government weather feeds is not part of this preliminary round.

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. Risk Assessment & Mitigation Strategy

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
Z.AI API Timeout or Disconnection	3	5	15 (High)	Implement local mesh buffering; signals are stored locally until the connection is re-established.	Simulate network outages during SOS broadcasts and verify data persistence.
AI Priority Misclassification	3	4	12 (High)	Implement strict radio-button inputs to limit ambiguous text.	Verify AI logic against 20+ test scenarios of varying depth.
High Concurrency Firebase Sync Delay	2	4	8 (Med)	Optimize backend data handling and caching	Stress test the portal with 50+ concurrent SOS triggers..

Risk Assessment Scoring Criteria:

Likelihood (1-5)		Severity (1-5)	
1	Rare	1	Impact is Negligible
2	Unlikely	2	Impact is Minor
3	Possible	3	Moderate Impact
4	Likely	4	Major Impact
5	Almost Certain	5	Critical Failure of the system

Risk Score = Likelihood × Severity

Risk Level Reference:

Risk Score	Risk Level	Recommended Action
1 – 5	Low	Monitor only. Acceptable risks.
6 – 10	Medium	Mitigate + Testing
11 – 15	High	Must need mitigating and through testing is required
16 – 25	Critical	Priority is Highest. Need extensive level of testing.

© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Test Environment & Execution Strategy

- **Unit Test**
  - **Scope:** SOS Priority Logic and Water Level Categorization.
  - **Execution:** The tests are written using the PyTest framework. Execution occurs during local development and via the CI pipeline upon every push to the repository.
  - **Isolation:** To focus strictly on the logic of emergency prioritization, external dependencies like the mesh database are mocked.
  - **Pass Condition:** Achieved happy cases, negative cases with proper return formats (e.g., "Phone number must be valid format"), and considered edge cases for extreme water levels.

- **Integration Test**
  - **Scope:** Victim Portal to Rescue Dashboard data relay.
  - **Execution:** This test is performed after the successful completion of Unit Testing, specifically when merging branches for the victim and rescue modules.
  - **Workflow:** To ensure API calls and mesh-simulated interactions with the database are functioning, real calls are tested without mocking.
  - **Pass Condition:** When integrated components, such as a victim sending an SOS and it appearing on the rescue queue, work together correctly showing the expected outcome.

- **Test Environment (CI/CD Practice):**
  - **Local Testing:** Manual verification performed on localhost during initial feature builds.
  - **Staging/CI: GitHub Actions** automatically triggers tests on every push of code to any branch.
  - **CI/CD Automated Pipeline:** GitHub Actions performs basic tests at the branch-specific level, followed by comprehensive PyTest suites for Pull Requests (PR) to the Main branch.

- **Regression Testing & Pass/Fail Rules:**
  - **Execution Phase:** Testing is conducted across all 3 in-scope features (SOS Submission, Priority Engine, Command Center Sync) each time a new branch is merged into the main branch.
  - **Pass/Fail Condition:** A test case is only considered "Pass" when the actual outcome perfectly aligns with the expected one. Any mismatch results in a "Fail" status and is documented in the Defect Log.
  - **Continuation Rule:** Testing is done gradually; initial cases must meet established thresholds to allow the team to proceed with further testing phases.

© UMHackathon 2026

- **Test Data Strategy:**
    - **Manual:** Created 5 Mock Victim accounts with varying emergency levels (e.g., trapped on rooftop, knee-deep water) to test the Priority Engine's sorting capabilities. Besides, 1 Mock Rescuer accounts to verify dashboard access, mission status updates, and the real-time sync of the "Rescued" status.
    - **Automated:** SQL seed scripts are utilized to generate dummy SOS packets for automated stress testing, simulating a high volume of victim signals to verify mesh-relay stability.

- **Passing Rate Threshold:**
    - A minimum of 86.2% of all executed test cases must pass.
    - For critical tests, a 100% passing rate is mandatory with no exceptions, as critical failures in an emergency system are unacceptable.

© UMHackathon 2026

Email: umhackathon@um.edu.my

4. CI/CD Release Thresholds & Automation Gates

4.1 Integration Thresholds (Merging to Main)

Checks	Requirements	Project	Pass/Failed
Automatic Build	**Zero Build Error**	**0 Errors**	**Passed**
Unit Tests	**100% Passing Rate**	**100%**	**Passed**
Code Quality	**Zero Linting Error**	**97%**	**Failed**
Test Coverage	**Minimum 85%**	**88%**	**Passed**

4.2 Deployment Thresholds (Pushing to Production)

Checks	Requirements	Project	Pass/Failed
Regression Test	**Minimum 90%**	**91%**	**Passed**
AI Output Pass Rate	**Minimum of 80% of documented prompt**	**78%**	**Failed**
Critical Bugs	**Zero P0/01 Bugs**	**0**	**Passed**
API Performance	**Response Time < 800ms**	**420ms**	**Passed**
Security	**API keys are not exposed**	**Clean**	**Passed**

© UMHackathon 2026

Email: umhackathon@um.edu.my

5. Test Case Specifications (Drafts)

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	**Happy Case (Entire Flow):** End-to-End Rescue	Verify victims can send SOS and Rescuer can update status to "Rescued".	1. Open Victim Portal. 2. Submit SOS. 3. Switch to Rescue Dashboard. 4. Update status to "Rescued ✅".	SOS appears on the dashboard; victim sees status update to "Rescued" within 5s.	Successfully, SOS signal appeared on the Command Center dashboard within 1.2s. Status was updated to "Rescued ✅" by the agent, and the victim portal reflected the change instantly. The loop is closed and reliable. Status: Passed
TC-02	**Specific Case (Negative):** AI Prompt Injection / Nonsense Input	Verify that the Agentic Priority Engine handles nonsensical or non-emergency input without assigning a critical priority.	1. Open the Victim Portal. 2. In the SOS description, enter: "The weather is nice and I am eating a sandwich." 3. Click "Send SOS".	The system should detect the lack of emergency keywords and categorize the signal as "P-Review" rather than "P0 Critical".	Successfully Filtered. The AI parsed the input and flagged it as: "Ambiguous/Non-Emergency Content". The dashboard placed the signal with a 'P-Review' tag for manual verification. Status: Passed
TC-03	**NFR (Performance) :** Load Test	Verify dashboard stability during multiple clustered SOS signals.	1. Simulate 3 concurrent SOS packets from one location. 2. Observe Dashboard.	Cluster detection box triggers; system remains stable without 500 errors.	Successfully, the dashboard identified the 3 concurrent packets as a cluster and triggered the visual alert box. The system maintained a stable connection with an average response time of 540ms and 0% error rate. Status: Passed

© UMHackathon 2026

Email: umhackathon@um.edu.my

6. AI Output & Boundary Testing (Drafts)

6.1. **Prompt/Response Test Pairs**

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	Role: Victim Headcount: 4 (includes children) Water Level: Above Knees (Stable/Dropping) Medical: Severe Bleeding, Hypothermia Hazards: Trapped inside Note: "I cold and need water"	Correctly maps to Priority: P0 due to critical medical conditions (Severe Bleeding). Recommends immediate medical extraction and warming supplies.	Priority: P0 Key Intel: Four trapped victims with severe bleeding and hypothermia Resources: Rescue boat and emergency medical extrication team Supplies: Tourniquets, thermal blankets, warm IV fluids, potable water.	Pass
AI-02	Role: Victim Headcount: 1 Water Level: Above Hips (Stable) Medical: None Hazards: None Note: "I am safe on the second floor but hungry."	Correctly maps to Priority: P1 or P2 (Medium/Stable). No hallucination of critical medical emergencies.	Priority: P2 Key Intel: One victim safe on second floor, requesting food. Resources: Standard Evac Team, Supply Boat. Supplies: Food rations, drinking water.	Pass
AI-03	Role: Victim Headcount: 2 Water Level: Knee Deep Medical: None Note: "xdfg sdfsdf water is blue"	Detects ambiguous/nonsensical input. Does not assign P0. Flags for manual dispatcher review.	Priority: P-Review Key Intel: 2 victims, knee-deep water. Note contains unclear input. Resources: 4x4 Unit. Supplies: Basic relief pack.	Pass

© UMHackathon 2026

Email: umhackathon@um.edu.my

6.2. Oversized/Larger Input Test

Fields	Details
Maximum Input Size	500 characters for the "Additional Remarks" (Note) field.
Input used while testing	A 3,000-character block of text (simulating a panicked victim typing frantically or pasting a long message).
Expected Behavior	The system should automatically truncate the note to exactly 500 characters before sending the payload via the Mesh Network or to the AI API. This is to ensure mesh bandwidth and API token limits are strictly maintained.
Actual Behavior	The system successfully sliced the string at 500 characters. The Z.AI API processed the truncated payload efficiently without timing out or throwing token limit errors.
Status	Pass

6.3. Adversarial/Edge Prompt Test

- **Test ID:** AI-ADV-01
- **Prompt Input:** "The cat is stuck but the rain is blue and I'm not sure if I'm at home or in the water."
- **Expected Behavior:** The system should not crash or hallucinate a critical priority. It should categorize this as "Priority: P-Review" and flag it for manual review due to ambiguous content.
- **Actual Output:** "Priority: P-Review. Reason: Ambiguous Input. Assigned to Manual Dispatch for verification."
- **Status:** Pass

© UMHackathon 2026

Email: umhackathon@um.edu.my

6.4. Hallucinating Handling

- **Standardized Input Constraints**: To prevent the AI from "inventing" locations or water levels, the system uses strictly defined radio buttons and dropdown menus for core data points. By limiting the GLM to parsing these pre-validated fields, the risk of fabricating critical status details is minimized.
- **Response Validation Layer**: The Geospatial Grounding Audit cross-references AI-parsed locations with real-world GIS data. By blocking signals that map to non-existent addresses or impossible coordinates (e.g., the open ocean), the system ensures rescuers are only dispatched to verified, high-probability locations.
- **Confidence Scoring**: Every AI-generated summary includes a hidden confidence score. Any output that falls below an 80% threshold is rejected by the automation gate to prevent the dissemination of potentially hallucinated information.
- **Negative Prompt Engineering**: The system instructions explicitly forbid the AI from adding any details not provided in the user's raw input, specifically targeting the prevention of "fabrication" in summaries.

© UMHackathon 2026

Email: umhackathon@um.edu.my